حلقه‌ها در کاتلین

```
fun main() {
    for (i in 1..3) {
        println("Kotlin")
    }
}
```

آموزش while و do-while وfor

بر اساس فصل چهارم کتاب راهنمای کاتلین

ارائه دهنده: امیرعباس بهمن آبادی

حلقه چیست؟

حلقه مثل یک دستگاه تکرار است :تا وقتی شرط درست (true) باشد، کد داخلش را بارها اجرا می‌کند و وقتی شرط false شد، متوقف می‌شود.

1

شرط بررسی می‌شود

تا وقتی true باشد، حلقه ادامه پیدا می‌کند

2

بدنه اجرا می‌شود

دستورات داخل حلقه یک دور دیگر تکرار می‌شوند

3

شرط false می شود

حلقه متوقف و برنامه ادامه پیدا می‌کند

یک میلیون بار چاپ، فقط با یک خط!

```
for (i in 1..1000000) println("Hello Kotlin")
```

حلقه while: اول شرط، بعد اجرا

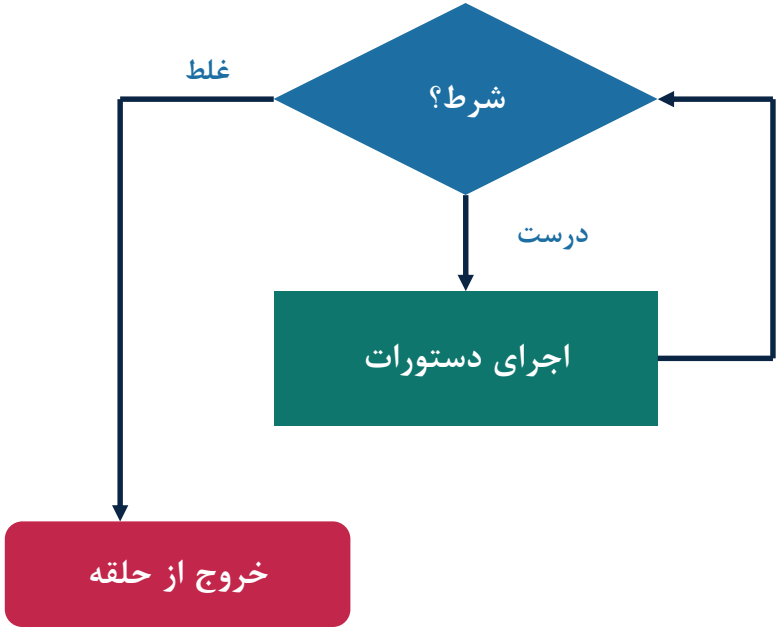
```
while (condition) {
    // statements
}
```

```
fun main() {
    var i = 0
    while (i < 5) {
        println(i)
        i++
    }
}
```

1. شرط بررسی می‌شود
2. اگر true بود، بدنه اجرا می‌شود
3. برمی‌گردیم و دوباره شرط را می‌سنجیم
4. با false شدن شرط، حلقه تمام می‌شود

```
output: 0 1 2 3 4
```

فلوچارت حلقه while



1. شرط مثلاً (i<5) بررسی می‌شود
2. درست: بدنه اجرا و به شرط برمی‌گردیم
3. غلط: از حلقه خارج می‌شویم

```
i: 0 1 2 3 4 | 5 = توقف
```

do-while و تفاوتش با while

While: اول بررسی، بعد اجرا

تا وقتی هوا سرد است لباس گرم بپوش؛ اگر هوا از اول گرم باشد، هیچ‌وقت اجرا نمی‌شود.

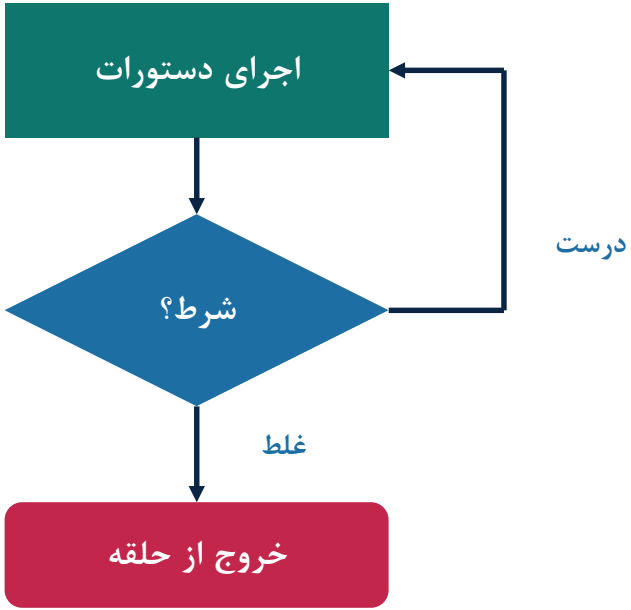
```
do {
    // statements
} while (condition)
```

do-while: اول اجرا، بعد بررسی

ماشین دما اول لباس گرم می‌دهد، بعد می‌پرسد هوا گرم شد؟ پس حداقل یک بار اجرا می‌شود.

```
fun main() {
    var i = 1
    do {
        println(i)
        i++
    } while (i <= 5)
}
```

فلوچارت حلقه do-while



1. بدنه حتماً یک بار اجرا می‌شود
2. بعد شرط بررسی می‌شود
3. درست: دوباره اجرا؛ غلط: خروج

حداقل یک بار اجرا

حلقه for با step و until

```
for (number in 0..10) {
    println(number)
}
```

شمارنده را خودش زیاد می‌کند و کدش از while کوتاه تر است.
«..»عدد آخر را هم شامل می‌شود: ۰ تا ۱۰

```
for (item in 0..20 step 3) {
    println(item)
}
```

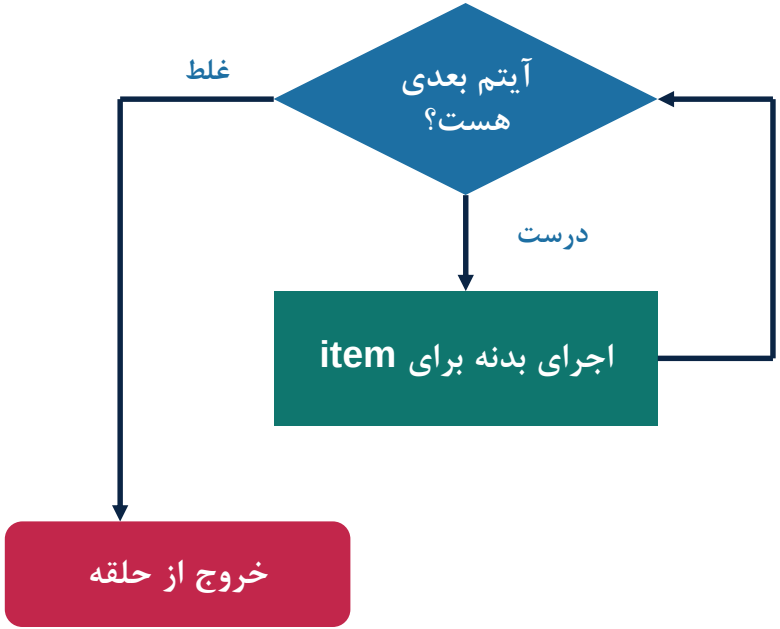
stepگام را تعیین می‌کند .خروجی: ۰،۳،۶،۹،۱۲،۱۵،۱۸

```
for (item in 0 until 21 step 3)
```

until عددآخر(۲۱) را شامل نمی‌شود.

نکته :هر کاری با for شدنی است با while هم شدنی است، ولی برعکسش همیشه نه؛ چون for شرط ندارد.

فلوچارت حلقه for



1. for آیتم بعدی بازه را برمی‌دارد
2. اگر آیتمی مانده، بدنه اجرا می‌شود
3. با تمام شدن بازه، حلقه پایان می‌یابد

بدون ++i دستی

مثال :جمع سه عدد با for

```
fun main() {
    var sum = 0
    for (item in 1..3) {
        val num = readLine()!!.toInt()
        sum += num
    }
    println("sum = $sum")
}
```

ورودی‌ها

5 10 20

35

مجموع سه عدد

هر دور حلقه یک عدد می‌خواند و به sum اضافه می‌کند.

پروژه: جمع و میانگین ۱۰ عدد

```
fun main() {
    var sum = 0
    for (item in 1..10) {
        val num = readLine()!!.toInt()
        sum += num
    }
    println("sum: $sum")
    println("mean: ${sum / 10}")
}
```

ورودی: 1 2 3 5 7 0 4 3 6 9

میانگین	جمع
4	40

توجه: تقسیم دو عدد int نتیجه‌ی صحیح می‌دهد. (40 / 10 = 4)

کنترل حلقه: break و continue

break: خروج کامل از حلقه

```
for (item in 0..10) {
    if (item == 5) break
    println(item)
}
```

```
output: 0 1 2 3 4
```

با رسیدن به 5حلقه کاملاً تمام می‌شود.

continue: رد کردن یک دور

```
for (item in 0..10) {
    if (item == 5) continue
    println(item)
}
```

```
output: 0 1 2 3 4 6 7 8 9 10
```

فقط دور 5رد می‌شود و حلقه ادامه می‌دهد.

خلاصه فصل چهار

نوع	زمان بررسی شرط	کاربرد
`while`	قبل از اجرای بدنه	تعداد تکرار نامشخص؛ شاید اصلاً اجرا نشود
`do-while`	بعد از اجرای بدنه	بدنه باید حداقل یک بار اجرا شود
`for`	خودکار روی بازه	تعداد تکرار یا مجموعه مشخص است
`break / continue`	داخل حلقه	رد کردن یک دور /توقف کامل

```
while (learning) {
    practice()
}
```

ممنون از توجه شما